# CARD: How to run and present the MVP

## 1. Start the project

This guide follows the current MVP. It covers setup, a suggested 8-10 minute demonstration, every main feature, and a complete new-evaluation example.

**What to say first:** "CARD is a local risk-assessment gateway for the external resources an AI coding agent wants to use. It scores packages, skills, MCP servers and URLs, explains the evidence, and returns allow, review or deny."

### Before the presentation

1. Install Node.js 22 or newer, which includes npm. Download or clone the CARD repository from https://github.com/FireFlamingo/MOSAIC.
2. Open a terminal in the CARD project folder. You should see package.json there. On this computer, the folder is:

   ```
   C:\Users\amogh\OneDrive\Desktop\Assignments\EDI5\CARD
   ```

3. Run these commands one at a time. The initial dependency installation needs internet access.

   ```
   npm ci
   ```

   ```
   npm run build
   ```

   ```
   npm start
   ```

4. Keep that terminal open. Open http://127.0.0.1:4318/ in your browser. Wait for "API reachable" in the header.
5. Open Policy. Set Hold for review to 20, Deny the request to 50, and Cross-artifact correlation to on. Click Save policy if you changed anything. These are the weighted-v3 defaults. Existing saved evaluations retain earlier models; submit or replay fresh requests.
6. Return to Overview. Rehearse once before presenting. Old observations remain saved; you do not need to clear them. Each replay creates a new session.

### On presentation day

If dependencies are installed and the build is current, only run npm start, then open the same browser address. The built app and bundled fonts work locally without internet. Rebuild after changing the source. Stop the server afterwards with Ctrl+C in its terminal.

For development only, npm run dev serves the interface on port 4317 and the API on 4318. Use the single-server build above for a simpler presentation.

# 2. Suggested live presentation sequence

**1. Introduce Overview (about 1 minute).** Show the total evaluations, allowed requests, pending reviews and denied requests. Explain that the supplied examples are synthetic inputs processed by the real local engine. The figures come from stored records, so totals depend on previous runs.

**2. Show a straightforward Allow (30 seconds).** Click Replay workflow, then Known publisher install. The new package scores 0 and is Allowed with the default policy. Explain: "No configured risk indicators matched the supplied evidence. This does not independently certify the package as safe."

**3. Demonstrate session correlation (about 2 minutes).** Replay Linked trust signals. The URL scores 50 and is Denied. The skill scores 30 and Needs review: provenance contributes 10 and the linked name contributes 20. Generic history adds no extra points because the stronger name signal covers that group. Open the skill to show the calculation and research reference.

**4. Show filtering and navigation (30 seconds).** Close the details. Use the session dropdown to choose a session. Click View session to see only that session's requests. In Requests, try the artifact-type buttons, decision dropdown and search box. Clear filters before moving on.

**5. Create an evaluation live (about 2 minutes).** Click New evaluation and enter page 3's example. Submit it, show the 20-point weighted breakdown and How the score adds up. Expand Supplied metadata & content and point out the session, model version, policy used and receipt.

**6. Resolve a review (about 1 minute).** Enter Reviewer note: "Presentation example: reviewed the supplied metadata." Click Allow request. Show the recorded review. In Review queue, it is no longer pending. Its original score remains 20. Replay Young integration needs review for another held request to demonstrate Deny request.

**7. Demonstrate policy (about 1 minute).** In Policy, move the preview slider: below 20 Allow, 20 to below 50 Needs review, 50-100 Deny. Turn correlation off and save; replay Linked trust signals. Its skill becomes 10 and Allowed. Restore correlation afterwards. Previewing creates no records; old results do not change.

**8. Finish with the audit trail (30 seconds).** Click Export audit trail, or Download audit on Overview. Show the downloaded card-audit.json file. Explain that records persist across restarts and that original evaluation receipts can be checked offline.

**Closing sentence:** "This MVP demonstrates one explainable scoring and review workflow across four artifact types. Automatic interception and independently verified evidence are work for the final project."

# 3. How to create a new evaluation

Use review 20, deny 50 and correlation on. Choose a fresh session ID, such as presentation-review-01. Use -02, -03 and so on for later rehearsals.

## Enter this complete example

1. Click **New evaluation** in the upper-right area. On a narrow screen, it appears below the page heading. The keyboard shortcut is N when you are not typing in a field.
2. Select **MCP server** under Artifact type. This selects the kind of resource being assessed; it does not connect to a real server.
3. Set **Artifact name** to presentation-notes.
4. Set **Session ID** to presentation-review-01, or another fresh ID.
5. Set **Source** to https://example.invalid/tools. This is a synthetic example address and is not fetched.
6. Leave **Artifact existence** as Unknown.
7. Set **Signature** to Reported unsigned.
8. Set **Age in days** to 6.
9. Set **Downloads** to 18.
10. Expand **Content & permissions**. Enter "Summarise the selected project notes." as Static content. Check only network:egress; leave the other permissions unchecked.
11. Click **Evaluate request**. The evaluation details should open automatically.

## What you should see

**Expected result: 20/100, Needs review.** Provenance raw severity is max(unknown 25, unsigned 25) = 25. Network raw severity is 25. Full capacity is 75 + 50 + 75 + 50 = 250. Each weighted contribution is 25/250 x 100 = 10. **10 + 10 = 20.** Unsigned is already covered, and age/downloads are context only. Assume default policy and no earlier session activity.

The details show each reason and its points, request context, the original decision, supplied metadata/content and the receipt hash. Enter a Reviewer note, then choose Allow request or Deny request. Without a non-empty note, the review buttons stay disabled.

## What the fields mean

Artifact name and Session ID are required. A session groups related requests; reuse an ID only when you want their history connected. Source is an optional location label. For Remote URL, fill URL to evaluate with a full URL; if omitted, the engine tries the artifact name as the URL.

Existence and signature values are reports supplied by you, not independently verified facts. Unknown means the information was not supplied. Age and downloads are optional non-negative whole numbers. Leave them blank when unknown: entering zero explicitly means an age of zero or zero downloads. Text and permissions are optional static evidence. Nothing submitted is installed, fetched or executed.

# 4. Feature checklist

**Four artifact types.** Package assesses a software library request. Skill assesses agent instructions or rules. MCP server assesses a proposed external tool server. Remote URL assesses a proposed URL. All use one request format and the same scoring/review workflow.

**Risk scoring and explanations.** Severity scale and weighted-average aggregation follow Zahan et al., IEEE Security & Privacy (2023). Take each group's strongest raw severity and normalize against the full applicable capacity: 250 for packages/skills/MCP, 300 for URLs. Group weights total 100%; scores need no cap. Age/downloads are context only. Grouping and classifications are CARD adaptations.

**Overview and totals.** See total, allowed, pending-review and denied counts. Click a total to open the corresponding request list. Overview includes the active policy summary, session trace, decision stream and review inbox.

**Session trace.** Choose a session from its dropdown, inspect individual events and open View session. The visual trace shows the latest five events; the request list contains the session's full recorded history. Related earlier requests can add points to later evaluations.

**Search and filters.** Requests supports artifact-type buttons, a decision dropdown, search by artifact name/source/session, a session filter, clear filters and pagination. Search is case-insensitive. N opens an evaluation, / focuses search, and Escape closes dialogs.

**Evidence inspector.** Open an artifact name, inspect arrow, trace node or review card. See How the score adds up: all group weights, raw ratings, zero contributions and the exact total. Also see paper links, model version, policy used, supplied evidence and receipt. Earlier models are labelled.

**Review inbox and queue.** The inbox highlights held requests; Review queue lists pending decisions. A required note accompanies each human Allow or Deny. The resolved verdict is displayed without changing the original score or receipt. A resolved review cannot be reviewed again through this UI.

**Policy editor.** Type thresholds or use sliders; review must be lower than deny. Save policy, discard edits, toggle correlation, and try a score in Decision preview. Changes apply to future evaluations; previewing alone creates no evaluation.

**Four replay workflows, with default policy.** Known publisher install: 0, Allowed. Young integration: 20, Needs review. Linked trust signals: URL 50, Denied, then skill 30, Needs review; correlation off gives skill 10, Allowed. Over-privileged unknown source: 50, Denied. Replays use the real API and fresh sessions.

**Storage, export and integration.** Evaluations, reviews and policy persist in data/state.json. Export the audit as JSON and verify the original evaluation chain offline. A local HTTP API supports evaluations, reviews, policy, replay, state retrieval, health and export; a cooperating caller must obey its decisions.

**Usability.** The layout adapts to desktop and mobile. Refresh loads the latest saved state. Failed submissions retain the form; Reconnect lets you retry. Replay supports keyboard arrows. Manrope headings and Public Sans interface text are bundled with the app.

# 5. Questions and quick fixes

## What the MVP does and does not prove

**Does Allow install or run the artifact?** No. CARD records a decision. It never executes the submitted artifact. An integrating agent or caller would have to use the decision before proceeding.

**Does it already protect all agent activity automatically?** No. Automatic command interception, agent adapters, MCP proxying and HTTP enforcement are not implemented in this MVP.

**Does it verify the registry, website or signature?** No. Metadata and text come from the caller. URL checks parse the address locally. Live registry lookups, reputation feeds and signature verification are future work.

**Is 50 a 50% chance of an attack?** No. It occupies half the possible weighted concern budget for the applicable checks. It is not an attack probability. Functional tests verify calculation and controls; empirical detection accuracy has not been measured.

**What is special about correlation?** Generic earlier activity has raw severity 25; a shared name strengthens the same group to 50. In the four-group profile, these contribute 10 or 20 weighted points. Both are not added. The latest 20 earlier session events and original decisions are considered. Uplift remains a hypothesis to validate.

**Are receipts digitally signed?** No. Original evaluations are SHA-256 hash-linked. Verification can detect changed content or broken links, but cannot authenticate the author or detect an entirely rewritten chain. Review and policy events are recorded separately and are not authenticated by that chain.

## If something goes wrong during the demonstration

**npm is not recognised:** install Node.js 22 or newer and reopen the terminal. **package.json cannot be found:** move into the CARD folder before running the commands.

**The browser will not open the app / Gateway offline:** check that npm start is still running, use http://127.0.0.1:4318/, then choose Reconnect or the header refresh button. If the terminal says the port is already in use, check whether an existing CARD server is already running.

**A result differs from this guide:** restore review 20, deny 50 and correlation on. Use exact values and a new session. Old records keep their earlier calculation; replay or submit fresh requests.

**The review queue is empty:** replay Young integration needs review, or submit the example on page 3. **A request is missing:** clear type, decision, search and session filters, then check the next page. **The look appears outdated:** rebuild the project and refresh the browser.

**To verify an export:** from the project folder run the following, replacing the quoted path with the actual downloaded file location:

```
npm run verify:audit -- "C:/path/to/card-audit.json"
```

For an optional pre-presentation check, run npm run check and npm run test:browser. Browser tests use installed Edge on Windows; on other platforms, install the test browser with npx playwright install chromium first. These tests use separate temporary data, leaving your saved presentation records intact.

# 6. Explain the weights to the panel

**Source of the numbers:** Zahan et al., "OpenSSF Scorecard: On the Path Toward Ecosystem-Wide Automated Security Metrics," IEEE Security & Privacy, 21(6), 76-88, 2023. DOI: 10.1109/MSEC.2023.3279773. Section II explicitly reports Low 2.5, Medium 5, High 7.5 and Critical 10. Read the accessible author manuscript at https://arxiv.org/html/2208.03412v3#S2.

**Why 25, 50 and 75?** These are raw severity ratings preserving the published weight ratios, not additive points. Current group maxima are 75 provenance, 50 capabilities, 75 content and 50 context. Their total 250 gives weights 30%, 20%, 30%, 20%. URLs additionally assess transport, giving total capacity 300.

**Why these categories?** Weak proxies such as uncertain provenance or encoding are Low. A trust-boundary concern or stronger linked identifier is Medium. Reported nonexistence, remote execution patterns or instruction overrides get conservative High priority. The papers support these concerns; CARD assigns the categories. There has been no independent expert validation yet.

**Why remove age/download penalties?** We found no basis for universal "under 7 days" or "under 50 downloads" boundaries. Popular projects can be compromised and new projects can be legitimate. These fields remain context for the reviewer.

**Why does the total stay within 100?** Each group rating is its matched severity divided by its declared maximum. The rating is between zero and one. Group weights sum to one, so 100 times the weighted average is between zero and 100 naturally. There is no clipping. Unmatched groups stay in the denominator and contribute zero.

**Explain ops-mirror:** missing severity 75 gives 75/250 x 100 = 30. Sensitive capability severity 50 gives 50/250 x 100 = 20. Content and context contribute zero. **30 + 20 + 0 + 0 = 50/100**. Maximum case: 75 + 50 + 75 + 50 = 250; 250/250 x 100 = 100.

**Why review 20 and deny 50?** These are operational choices: review a medium concern in the four-group profile or two low concerns; deny when half the possible concern budget is occupied. The policy is not a learned optimum. A single medium URL cue is 16.67, so stricter URL deployments need a lower boundary or additional mandatory controls.

**Presentation wording:** "We use published severity ratios and normalized weighted-average aggregation. Fixed group ranges produce weights totaling 100%. Each matched concern contributes its weighted share, and the displayed parts add exactly to the final score. The group's definitions and decision policy are documented adaptations; final detection accuracy requires benchmark validation."

**Supporting conference papers:** Spracklen et al., USENIX Security 2025, on hallucinated packages; Ohm et al., DIMVA 2020, on malicious supply-chain attacks; Sejfia and Schäfer, ACM/IEEE ICSE 2022, on malicious npm detection; Abdelnabi et al., ACM AISec 2023, on indirect prompt injection. Complete references and every signal's rationale are in CARD-Scoring-Rationale.pdf and docs/SCORING.md.